C-PROGRAMING (CSA0202)

CO1 ASSIGNMENT – 1

Grama Jyothi Rural Electricity Co-operative
Electricity Billing System in C

NAME: LEELAVATHI R
REG.NO:1925260191

TASK 1 – Understanding the Problem and Identifying the Data

Grama Jyothi must calculate the monthly electricity bill for each household. The meter reader supplies the previous and current meter readings. The program calculates consumption, applies the tariff slabs, adds the fixed charge and prints the amount payable.

Inputs and outputs

Data item	Meaning and reason for storing it
Household number	Identifies the customer whose bill is being calculated. It may be stored as an integer if it is purely numeric; a character array is safer if IDs can contain letters or leading zeroes.
Category	Identifies the household tariff category, if different categories exist. A small numeric code or enum can represent a fixed set of categories.
Previous reading	The meter value at the start of the billing period. It must be retained to calculate consumption.
Current reading	The meter value at the end of the billing period. It is compared with the previous reading, including meter rollover.
Units consumed	Difference between readings, adjusted for rollover. It is a non-negative whole number for the stated integer meter.
Energy charge	Charge calculated from the consumption slabs. It is stored in paise to avoid decimal currency errors.
Fixed charge	A fixed amount added to every bill, independent of consumption.
Total bill	Sum of fixed charge and energy charge, represented in paise and displayed as rupees and paise.

Constants, rules and assumptions

- Constants include the meter maximum value (65,535), slab boundaries (100 and 200 units), slab rates (0, 225 and 450 paise per unit), and fixed charge (5,000 paise).
- Meter readings and units are non-negative whole numbers. A 16-bit meter register ranges from 0 to 65,535.
- Money may contain paise, so calculations must preserve fractional rupees exactly. The design stores money as integer paise.
- A bill must never be negative. Total bill must equal fixed charge plus energy charge. Each unit must be charged in exactly one slab.
- Assumption: one rollover can occur in a billing period. For a single rollover, consumption is (65,536 − previous) + current. Multiple rollovers would require additional information such as rollover count or interval readings.
- Assumption: the example tariff applies to all households. If categories have different tariffs, a tariff table must be selected using the category.

Text representation and complexity

If readings were stored as strings such as "01530", each reading would require d characters plus a terminating '\0'. Conversion with strtoul() takes O(d) time, and comparing two strings can take O(d) time in the worst case. A fixed-width character array uses approximately d + 1 bytes per reading. An integer reading uses a fixed number of bytes (typically 2 bytes for the specified 16-bit meter_t) and arithmetic/comparison takes O(1) time. Text preserves leading zeroes and is useful for raw input, but integer storage is more efficient for calculations.

For N households, storing numeric readings requires O(N) space overall; storing strings also requires O(N) space, but with a larger per-reading constant and O(d) conversion work per reading.

TASK 2 – Choosing the Data Types

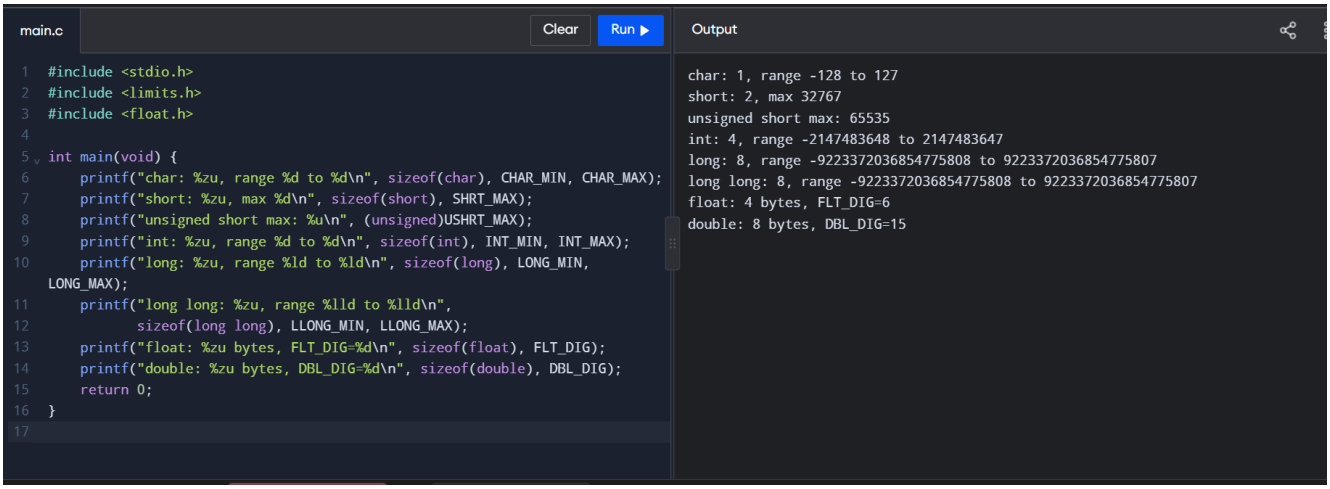
The following choices suit the stated 16-bit meter and the growth from 4,000 to 800,000 households. Exact sizes are implementation-dependent, so sizeof and the limits macros must be checked on the target compiler.

Data item	Suggested C type	Reason
Meter reading	unsigned short (typedef unsigned short meter_t)	The assignment specifies a 16-bit register. It can represent 0 through 65,535 when USHRT_MAX is 65,535.
Units consumed	unsigned int	Consumption is non-negative and can be larger than one meter reading after rollover handling.
Household number	unsigned int or char[]	Use unsigned int for numeric IDs; use a string if IDs may contain leading zeroes or letters.
Category	enum or int	A small, fixed set of categories is represented clearly by enum.
Slab rates / fixed charge	long or long long integer paise	Integer paise avoids binary floating-point currency errors.
District-wide total	long long	Provides a large integer range for accumulating many bills.
Tariff percentage	integer basis points or rational numerator/denominator	Avoids floating-point calculations where exact currency is required.

Sizes and ranges

On a common GCC system, sizeof(char) is 1 byte, sizeof(short) is 2 bytes, sizeof(int) is 4 bytes, sizeof(long) is 8 bytes on 64-bit Linux (but often 4 bytes on 64-bit Windows), and sizeof(long long) is 8 bytes. float is commonly 4 bytes and double 8 bytes. Use sizeof(type) to measure the actual system.

```
#include <stdio.h>
#include <limits.h>
#include <float.h>
int main(void) {
   printf("char: %zu, range %d to %d\n", sizeof(char), CHAR_MIN, CHAR_MAX);
   printf("short: %zu, max %d\n", sizeof(short), SHRT_MAX);
   printf("unsigned short max: %u\n", (unsigned)USHRT_MAX);
   printf("int: %zu, range %d to %d\n", sizeof(int), INT_MIN, INT_MAX);
   printf("long: %zu, range %ld to %ld\n", sizeof(long), LONG_MIN, LONG_MAX);
   printf("long long: %zu, range %lld to %lld\n",
        sizeof(long long), LLONG_MIN, LLONG_MAX);
   printf("float: %zu bytes, FLT_DIG=%d\n", sizeof(float), FLT_DIG);
   printf("double: %zu bytes, DBL_DIG=%d\n", sizeof(double), DBL_DIG);
   return 0;
}
```



Why money is stored as paise

A float or double stores most decimal fractions as binary approximations. Repeated additions can therefore produce a value slightly above or below the mathematically exact amount. Store Rs 162.50 as 16,250 paise instead. Integer addition is exact as long as the selected integer type does not overflow.

Example of floating-point accumulation: adding 0.1 as a binary floating-point value repeatedly may produce a result such as 399.999... rather than exactly 400.00. Across 4,000 bills, a small per-bill representation error can accumulate. Integer paise gives the exact total in paise.

Rollover calculation – completed TODO

Unsigned 16-bit arithmetic wraps modulo 65,536. Therefore subtracting the previous reading from the current reading in the unsigned meter type automatically gives the correct difference for one rollover.

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current) {
    return (meter_t)(current - previous);
}

int main(void) {
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

main.c

Clear Run ▶

```c
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current) {
    return (meter_t)(current - previous);
}

int main(void) {
    printf("sizeof(meter_t) = %zu bytes, maximum reading = %u\n",
           sizeof(meter_t), (unsigned)USHRT_MAX);
    printf("Consumption: %u units\n", consumption(1200, 1530));
    printf("Consumption: %u units\n", consumption(65400, 120));
    return 0;
}
```

Output

```
sizeof(meter_t) = 2 bytes, maximum reading = 65535
Consumption: 330 units
Consumption: 256 units
```

The second result is 256 because the register counts 135 values from 65,400 through 65,535 and then 121 values from 0 through 120. The total is 135 + 121 = 256. Unsigned arithmetic is modulo 65,536, so it does not produce a negative number.

If money were stored as double rupees, additions would still be O(1) time, and a double commonly uses 8 bytes. Rounding should be applied at a clearly defined point, normally to the nearest paise for each bill before adding bills. Rounding only the final total can make the sum differ from the sum of individually payable bills. Integer paise uses typically 8 bytes for long long and has exact addition; floating-point error can accumulate with n additions.

TASK 3 – Define the Constants

Tariff values must be declared once so that a revision does not require editing several parts of the program. The assignment uses macros for slab sizes, rates and fixed charge.

Completed billing function

```
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

long compute_bill(int units) {
    long paise = FIXED_CHARGE_PAISE;

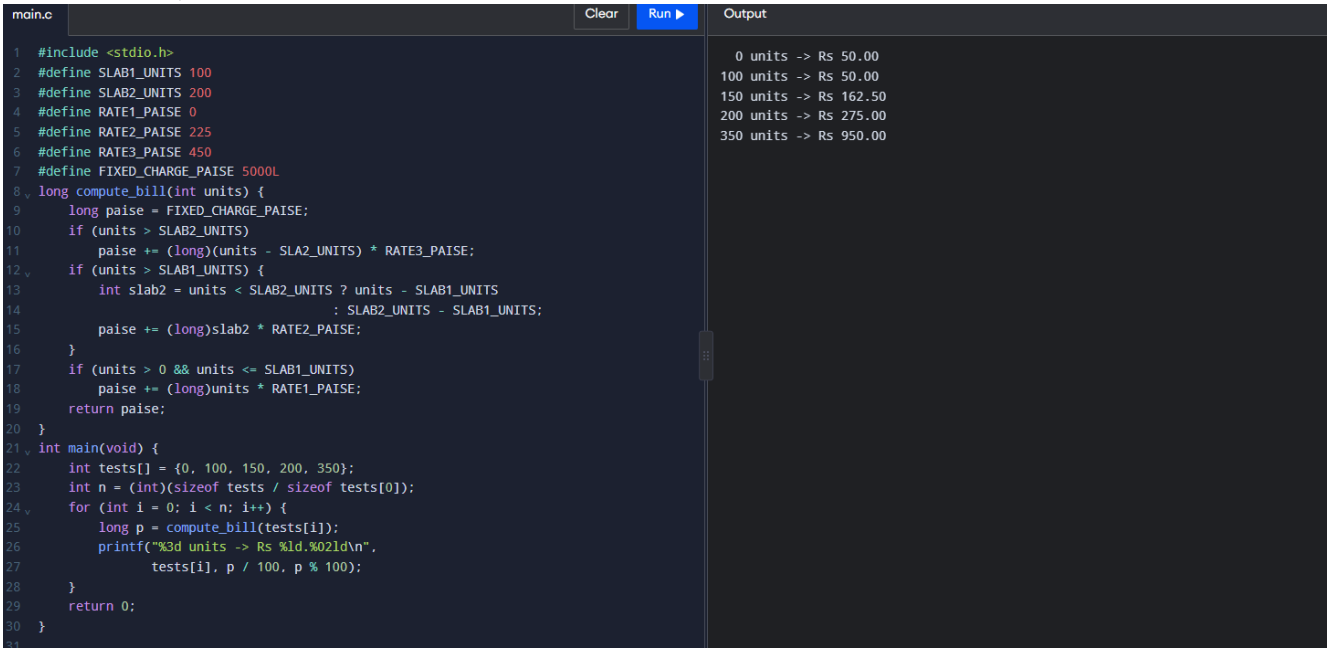
    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;

    if (units > SLAB1_UNITS) {
        int slab2 = units < SLAB2_UNITS ? units - SLAB1_UNITS
                                        : SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2 * RATE2_PAISE;
    }

    if (units > 0 && units <= SLAB1_UNITS)
        paise += (long)units * RATE1_PAISE;

    return paise;
}

int main(void) {
    int tests[] = {0, 100, 150, 200, 350};
    int n = (int)(sizeof tests / sizeof tests[0]);
    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
```



For 150 units, the first 100 units are free and the next 50 units cost 50 × Rs 2.25 = Rs 112.50. Adding the Rs 50 fixed charge gives Rs 162.50. For 350 units, the charge is 100 × Rs 2.25 + 150 × Rs 4.50 + Rs 50 = Rs 950.

Changing RATE2_PAISE from 225 to 230 changes the second-slab charge consistently because every calculation uses the same macro. The tariff change is made in one location.

#define, const and enum

- #define is processed by the preprocessor before compilation. It has no C type and normally occupies no storage. It is suitable for simple compile-time values used throughout the program.
- A const variable has a type, follows normal scope rules and can be inspected more easily in a debugger. The compiler may optimize it, but it is a typed object.
- enum is suitable for named integral choices such as HOUSEHOLD, COMMERCIAL and AGRICULTURAL categories. It is not suitable for fractional tariff values.
- Use uppercase names with underscores for macros, such as FIXED_CHARGE_PAISE. Running time is generally unaffected after optimization; the main differences are type checking, scope, debugging visibility and storage semantics.

TASK 4 – Use Macros Safely

Macros can shorten repeated expressions, but every parameter and the whole expression should be enclosed in parentheses.

```
#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
```

Sample:

```c
#include <stdio.h>

#define RUPEES_TO_PAISE(r) ((long long)(r) * 100LL)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))

int main()
{
    printf("Rupees to Paise: %lld\n", RUPEES_TO_PAISE(5));
    printf("Square: %d\n", SQUARE(4));
    printf("Maximum: %d\n", MAX(10, 20));

    return 0;
}
```

```
Rupees to Paise: 500
Square: 16
Maximum: 20
```

If SQUARE is written as #define SQUARE(x) x*x, then SQUARE(a+1) expands to a+1*a+1. For a = 2, this becomes 2 + 1×2 + 1 = 5, while the correct square is 9. With parentheses, it expands to ((a+1)*(a+1)).

MAX(a,b) may evaluate an argument more than once. For example, MAX(i++, j) can increment i during the comparison and again when the selected expression is evaluated. This side effect makes the result difficult to predict. Do not pass expressions with side effects to such macros.

A function is preferable when type checking, single evaluation, debugging or safer argument handling is important. A static inline function provides function-like type checking and usually allows the compiler to remove call overhead.

```
static inline int square_int(int x) {
    return x * x;
}
static inline int max_int(int a, int b) {
    return (a > b) ? a : b;
}
```

A macro is expanded at each of its k uses, which can increase generated code size roughly with the amount of expanded code. An inline function may be inlined by the compiler or emitted once as a function. A macro has no call overhead after expansion; an ordinary function call may have a small call/return cost, while an optimized inline function can have similar runtime cost to a macro.

To inspect macro expansion, save the program as billing.c and run: gcc -E billing.c -o billing.i. The .i file shows the preprocessed source after macro expansion and header inclusion.

TASK 5 – Organise the Program with Header Files and Conditional Compilation

A maintainable program separates declarations from implementation. A header file contains constants, type definitions and function prototypes. A .c file contains function definitions and main().

Example header file: billing.h

```
#ifndef BILLING_H
#define BILLING_H

#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L

typedef unsigned short meter_t;

unsigned int consumption(meter_t previous, meter_t current);
long compute_bill(int units);

#endif
```

Example implementation with DEBUG mode

```
#include <stdio.h>
#include "billing.h"

unsigned int consumption(meter_t previous, meter_t current) {
    unsigned int units = (meter_t)(current - previous);
#ifdef DEBUG
    fprintf(stderr, "[DEBUG] previous=%u current=%u units=%u\\n",
            (unsigned)previous, (unsigned)current, units);
#endif
    return units;
}
```

The include guard prevents the same header from being processed more than once in a translation unit. Without it, repeated inclusion may cause duplicate declarations or definitions. Compile normally with gcc billing.c main.c -o billing. Enable debugging with gcc -DDEBUG billing.c main.c -o billing. Debug messages are included only when DEBUG is defined.

- #include <stdio.h> searches the compiler's system include directories for a standard or installed header.
- #include "billing.h" searches the current source/project location first and is normally used for the programmer's own header.

If debug output uses a run-time variable, every relevant call checks if (debug). This adds a small O(1) condition test per call, and the debug code remains in the executable. With #ifdef DEBUG, the disabled debug statements are removed before compilation, so there is no run-time test or disabled debug-code footprint in the executable.

TASK 6 – Read and Print Data Correctly

The format specifier must match the type passed to scanf or printf. A mismatch can cause undefined behaviour, incorrect output or memory corruption.

Type	printf specifier	scanf specifier
int	%d	%d
unsigned int	%u	%u
long	%ld	%ld
long long	%lld	%lld
float	%f	%f
double	%f	%lf
char	%c	%c
char array/string	%s	%s

For printf, a double is printed with %f. For scanf, a double requires %lf, while a float requires %f. A long requires %ld. Using %d for a long is incorrect because the function will interpret the argument using the wrong type.

Input validation

```
int previous, current;
printf("Enter previous and current readings: ");
if (scanf("%d %d", &previous, &current) != 2) {
    printf("Invalid input. Enter two whole numbers.\\n");
    return 1;
}
if (previous < 0 || previous > 65535 ||
    current < 0 || current > 65535) {
    printf("Reading is outside the meter range.\\n");
    return 1;
}
```

scanf returns the number of input items successfully assigned. Checking for exactly two assignments detects letters or missing values. The program should also validate the meter range and determine whether a lower current reading is a legitimate rollover or an invalid reading.

Printing rupees and paise

```
long amount_paise = 16250;
printf("Bill: Rs %ld.%02ld\\n",
       amount_paise / 100, amount_paise % 100);
```

The output is Rs 162.50. Integer division obtains the rupee part, and the remainder obtains paise. The %02ld format prints paise using two digits.

An alternative is to read a complete line with fgets() and then parse it using strtol() or sscanf(). Processing a line of length L takes O(L) time and needs a buffer of O(L) space (or a fixed maximum buffer). fgets() makes it easier to discard or report a complete invalid line and avoids scanf's leftover-input problems. Direct scanf() is concise but recovery from malformed input needs careful handling.

TASK 7 – Control Type Conversion and Precision

C automatically converts values in mixed-type expressions. Small integer types such as unsigned short are usually promoted to int or unsigned int before arithmetic. The usual arithmetic conversions then determine the common type. Explicit casts should be used only when the intended conversion is safe.

Examples of unexpected results

```
/* 1. Integer division */
printf("%d\\n", 5 / 2);          /* prints 2, not 2.5 */

/* 2. Integer division before assignment */
double x = 5 / 2;               /* x becomes 2.0 */
double y = 5.0 / 2;             /* y becomes 2.5 */

/* 3. Unsigned wraparound */
unsigned short a = 0;
unsigned short b = 65535;
printf("%u\\n", (unsigned short)(a - b)); /* 1 modulo 65536 */
```

The first two examples show that assigning an integer-division result to double does not restore the discarded fraction. At least one operand must be converted to floating point before division. The third example demonstrates modulo arithmetic for an unsigned 16-bit value.

Casting a large long or long long value to int can discard high-order bits or produce an implementation-defined result if the value is not representable. Do not narrow district totals to int. Keep the total in long long.

For currency, use integer paise. If a percentage or fractional tariff must be applied, use an integer rational calculation and define a rounding rule. For example, for a 20% surcharge, calculate base_paise × 120 / 100 and round to the nearest paise using an explicit formula, while checking for multiplication overflow.

If all calculations used float and were rounded only at the end, each bill and the total could contain representation error. The error may accumulate as n increases. Both float and integer addition are O(1) per value; float commonly occupies 4 bytes, while a long long commonly occupies 8 bytes. Integer paise is exact within range and is therefore preferred for bills.

TASK 8 – Trace the Compilation Process

A C program passes through preprocessing, compilation, assembly and linking. Each stage produces a different output.

Command	Stage and output
gcc -E billing.c -o billing.i	Preprocessing: expands macros, includes header contents and processes conditional compilation.
gcc -S billing.c -o billing.s	Compilation: translates C into assembly language.
gcc -c billing.c -o billing.o	Assembly: produces an object file containing machine code and symbol information; it is not yet a complete executable.
gcc billing.o main.o -o billing	Linking: combines object files and required libraries to create the executable.

The preprocessor replaces macro names with their definitions, inserts included header contents, and removes branches excluded by #ifdef or #ifndef. It does not perform normal C type checking.

- Compile-time error: using an undeclared variable or missing a required semicolon. The compiler reports the source error.
- Link-time error: calling a function whose definition is missing, such as declaring compute_bill() but not linking the file that defines it. The linker reports an undefined reference.
- Run-time error: invalid input, division by zero, or an out-of-range access that occurs while the executable runs.

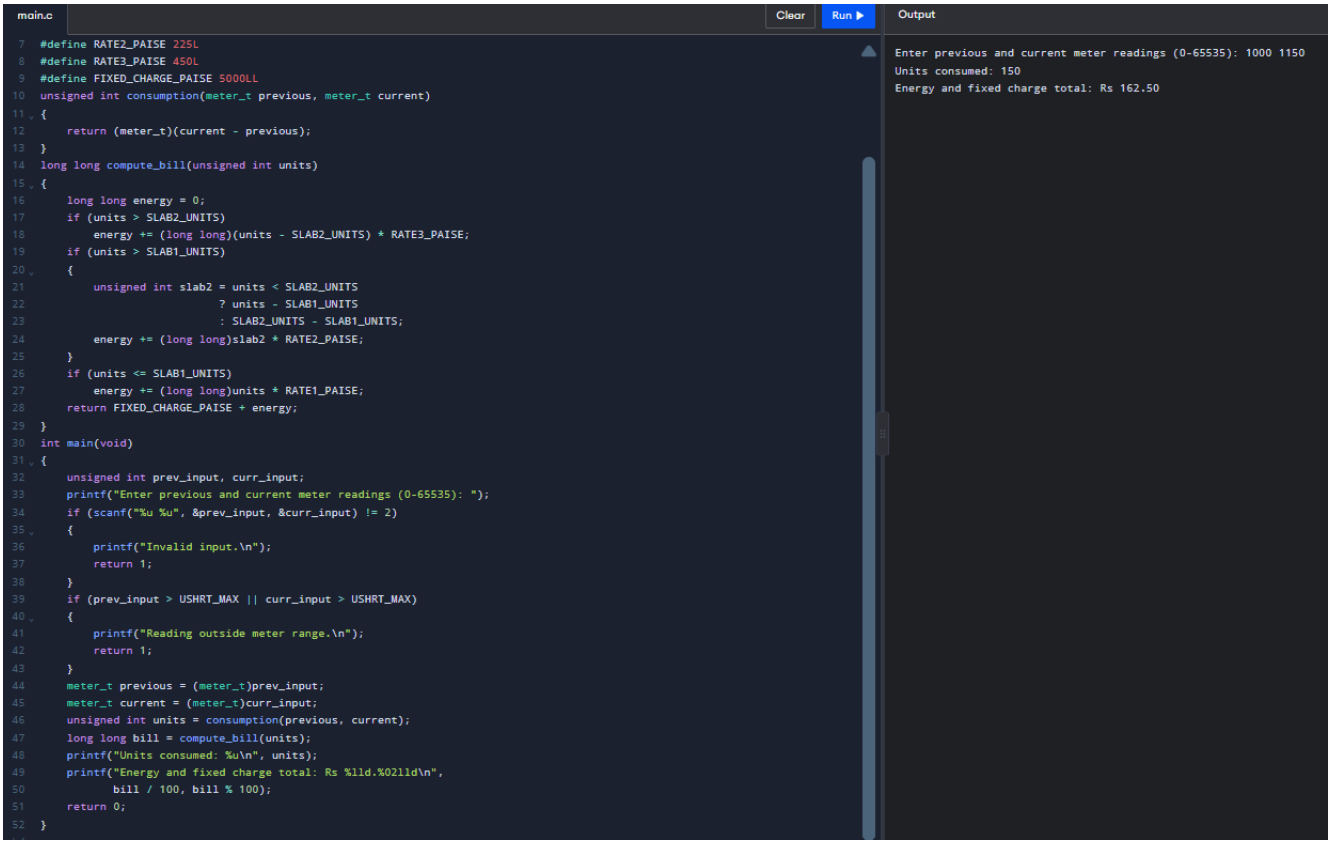
Compile with gcc -Wall -Wextra to receive useful warnings. Typical warnings include an unused variable, a signed/unsigned comparison, a format-specifier mismatch or a missing return. Resolve them by removing unused variables, matching types and format specifiers, validating input and returning a value from every non-void function.

For a multi-file program (input.c, tariff.c and report.c), each source file is compiled separately to an object file and then linked with the shared header declarations. After changing one .c file, only that file normally needs recompilation before relinking. In a single-file build, the whole source file is recompiled after a change. Object files occupy additional disk space, but modular compilation can reduce rebuild time for large projects.

TASK 9 – Implement and Test the Billing Program

The following complete program calculates one bill from two readings. It uses the 16-bit rollover rule, integer paise, slab constants and input validation. A lower current reading is treated as a rollover in this simplified program; a production system should use meter event data or plausibility limits to distinguish rollover from a faulty reading.

```
#include <stdio.h>
#include <limits.h>
typedef unsigned short meter_t;
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0L
#define RATE2_PAISE 225L
#define RATE3_PAISE 450L
#define FIXED_CHARGE_PAISE 5000LL
unsigned int consumption(meter_t previous, meter_t current)
{
    return (meter_t)(current - previous);
}
long long compute_bill(unsigned int units)
{
    long long energy = 0;
    if (units > SLAB2_UNITS)
        energy += (long long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS)
    {
        unsigned int slab2 = units < SLAB2_UNITS
                           ? units - SLAB1_UNITS
                           : SLAB2_UNITS - SLAB1_UNITS;
        energy += (long long)slab2 * RATE2_PAISE;
    }
    if (units <= SLAB1_UNITS)
        energy += (long long)units * RATE1_PAISE;
    return FIXED_CHARGE_PAISE + energy;
}
int main(void)
{
    unsigned int prev_input, curr_input;
    printf("Enter previous and current meter readings (0-65535): ");
    if (scanf("%u %u", &prev_input, &curr_input) != 2)
    {
        printf("Invalid input.\n");
        return 1;
    }
    if (prev_input > USHRT_MAX || curr_input > USHRT_MAX)
    {
        printf("Reading outside meter range.\n");
        return 1;
    }
    meter_t previous = (meter_t)prev_input;
    meter_t current = (meter_t)curr_input;
    unsigned int units = consumption(previous, current);
    long long bill = compute_bill(units);
    printf("Units consumed: %u\n", units);
    printf("Energy and fixed charge total: Rs %lld.%02lld\n",
           bill / 100, bill % 100);
    return 0;
}
```



Test cases

Test	Input / units	Expected result
Zero consumption	0 units	Rs 50.00
First slab boundary	100 units	Rs 50.00
Second slab example	150 units	Rs 162.50
Second slab boundary	200 units	Rs 275.00
Above second slab	350 units	Rs 950.00
Meter rollover	65400 to 120	256 units; Rs 176.00
Letters	abc 120	Rejected: invalid input
Out-of-range reading	70000 120	Rejected: outside 0–65535
Negative reading	-1 120	Should be rejected; use a signed input check before conversion in production
Lower reading without rollover	1200 to 1100	Modulo calculation treats it as rollover; production validation must distinguish an implausible rollover from a faulty reading

For 256 units, the energy charge is 100 × Rs 2.25 + 56 × Rs 4.50 = Rs 477.00; with the Rs 50 fixed charge, the bill is Rs 527.00. The table's rollover bill is therefore Rs 527.00.

A hand-checkable set of five bills at 0, 100, 150, 200 and 350 units is Rs 50.00 + Rs 50.00 + Rs 162.50 + Rs 275.00 + Rs 950.00 = Rs 1,487.50. The total is 148,750 paise.

For a scalable tariff design, slab limits and rates can be stored in arrays and processed in a loop. The time per bill is O(s), where s is the number of slabs, and the arrays require O(s) extra storage. Fixed if statements have constant time for a fixed tariff structure, while arrays are easier to revise or load dynamically.

TASK 10 – Compare Expectations with Execution

A district may contain 800,000 households, so the total must be calculated in a type with enough range. The exact comparison depends on the actual average bill and the target system's integer sizes.

Illustration using the five test bills from Task 9: the average is Rs 297.50 per bill. At this illustrative average, the totals are:

Number of bills	Illustrative total	Total in paise
4,000	Rs 11,90,000.00	119,000,000
60,000	Rs 1,78,50,000.00	1,785,000,000
800,000	Rs 23,80,00,000.00	23,800,000,000

These are illustrative estimates based on the average of the five sample bills, not measured totals for real households. The actual total must be obtained by summing the actual bills.

A signed 32-bit int typically has maximum 2,147,483,647. It can hold 119,000,000 paise for 4,000 bills, but not 1,785,000,000? It does fit that value, while 23,800,000,000 paise exceeds the limit. A 32-bit long has the same limitation. A 64-bit long long has a much larger range and safely holds the illustrative district total.

If the accumulated total is held in double, it can represent very large magnitudes, but not every integer exactly once values exceed its exact-integer precision limit (commonly 2^53). The illustrative district total in paise is 23.8 billion, below 2^53, so a double can represent that integer exactly on common IEEE-754 systems. However, decimal fractions and repeated floating-point operations can still introduce precision concerns. long long provides exact integer paise within its range.

An addition is O(1) for int, long long and double. Typical storage is 4 bytes for int and 8 bytes for long long or double. Integer types fail by overflow when the range is exceeded; floating-point types lose exact integer or fractional precision at sufficiently large values.

TASK 11 – Recommend and Defend the Final Design

The recommended original billing design uses unsigned short for the 16-bit meter register, unsigned int for units consumed, long long for paise amounts and district totals, and named constants for slab boundaries, rates and fixed charge.

- Use integer paise for all bill calculations. This prevents decimal floating-point rounding errors in normal currency operations.
- Use unsigned arithmetic only for the defined meter rollover calculation. Validate raw input before converting it to meter_t.
- Use long long for accumulated totals so the district-scale sum does not overflow a typical 32-bit int.
- Keep tariff definitions in one header or tariff module. A revision is made in one place and the program is rebuilt and tested.
- Use include guards, function prototypes and a conditional DEBUG build to improve maintainability and testing.

Limitations: the simple design assumes one rollover at most per billing period, a common tariff for all customers and a fixed slab structure. It does not by itself handle meter faults, missing readings, multiple rollovers, category-specific tariffs, time-of-day pricing or audit trails. These require additional validation and data structures.

At district scale, the chosen long long total remains suitable for the illustrated totals, but the implementation should still check for overflow and test with realistic maximum consumption and tariff values.

If tariff values are loaded from a configuration file at start-up, the program can read t tariff values in O(t) time and store them in O(t) space. This avoids recompiling and redistributing the executable for every tariff revision. The program must validate the file, reject missing or invalid values and record the effective tariff version. Compile-time constants are simpler and safer for an unchanged tariff, but configuration files are more flexible for frequent revisions.

TASK 12 – Re-evaluate and Adapt the Design

Recommendation: MODIFY the original design. The original integer-paise principle remains useful, but the meter data model, tariff calculation and configuration mechanism must be expanded for smart-meter readings.

A. Impact Analysis

- The original system receives two cumulative meter readings per household for a monthly billing period. The new system receives a watt-hour reading every 15 minutes, giving 35,040 readings per house per year. For 800,000 houses, this is 28,032,000,000 readings per year, so storage, processing and data transfer become major design concerns.
- The original 16-bit unsigned short meter type is no longer sufficient as a general representation for yearly cumulative energy or large watt-hour totals. Use a wider integer type such as uint64_t for cumulative watt-hours and carefully chosen timestamp types. A single interval reading may fit in a smaller type, but yearly totals and aggregation need wider capacity.
- The old fixed #define tariff values are not suitable for values that the board wants to change without recompiling. Load tariff rates, slab limits and effective dates from a validated configuration file or database when the program starts. Keep only structural constants, such as the number of seconds in an hour or record-format limits, as compile-time constants.

The time-of-day tariff requires each interval reading to be associated with a timestamp. Consumption between 6 PM and 10 PM must be identified and charged at 20% more. If an interval crosses a tariff boundary, split the interval consumption proportionally only if the meter provides sufficient time resolution; otherwise use finer-grained meter data or a documented allocation rule. Store energy in watt-hours or integer subunits and money in integer paise, applying a defined rounding rule.

The new system should process readings in batches or a streaming pipeline, validate timestamps and duplicate/missing records, preserve an auditable reading history, and aggregate consumption by tariff period. The original design is therefore modified rather than completely discarded: exact integer money and careful validation are retained, while the meter representation, tariff source and processing architecture are upgraded.

Conclusion

The Grama Jyothi billing system should use appropriate integer types, integer paise, centralized tariff definitions, safe macros, validated input and modular compilation. The original design is accurate for monthly cumulative readings when its assumptions are satisfied. The smart-meter requirement needs wider cumulative counters, timestamp-aware time-of-day calculations and externally configurable tariffs.